

STAT 240

Week 3: Basic Stats and more R

Owen G. Ward

Today

- Basic statistics review
- Some more R
 - Data Visualization
 - Functions
 - Reading JSON files, viewing images

Midterm Update

- Midterm will be in class on **Wednesday February 25th**
- Much bigger room, more space for you

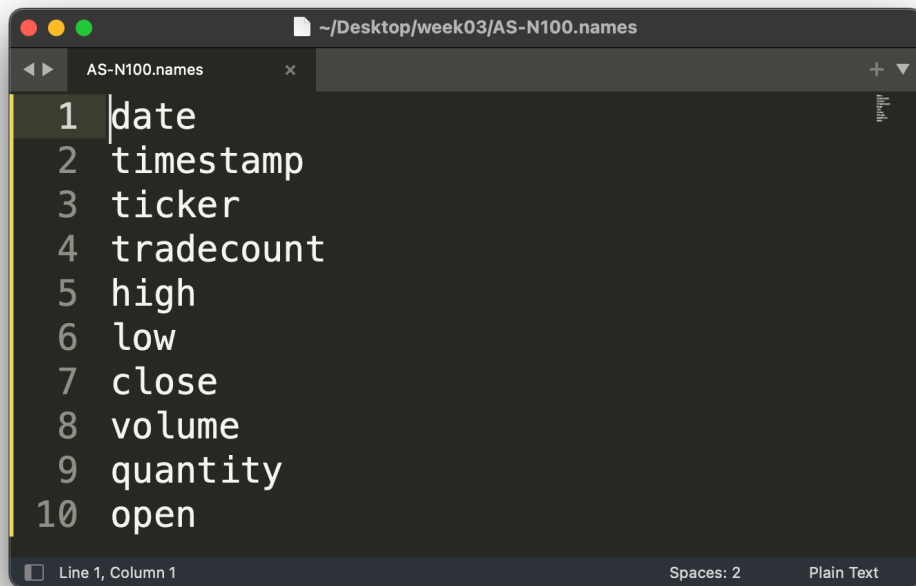
Statistics Refresher

Statistics review

- Five basic summary statistics: mean, median, standard deviation, min, max. In R these are the functions `mean`, `median`, `sd`, `min`, `max`

Example: NASDAQ

- We'll give some examples on the NASDAQ dataset. Here we consider a version of the dataset with a *.data* and *.name* file, like in Lab 01 (this Friday)



-
- We read in the data and the column names. This *.data* file does not have column names in it

```
stock_data <- read.table("AS-N100.data", header = FALSE)
columns <- readLines("AS-N100.names")
print(stock_data[1, "ticker"])
```

NULL

- There are normally several ways to read in data, these are two (older) options

-
- We can attach headers by using the `colnames` function:

```
colnames(stock_data) <- columns
print(stock_data[1, "ticker"])
```

```
[1] "LLTC"
```

Min and max

- Recall that the NASDAQ data contains minute-resolution prices for 27 stocks for one day. We'll extract the minimum opening price for GOOGL, using logical indexing.

```
colnames(stock_data) <- columns
google_stock <- stock_data[stock_data[, "ticker"] == "GOOGL", ]
print(min(google_stock[, "open"]))
```

```
[1] 1
```

- We can (hopefully) tell that something has gone wrong: These stock prices are supposed to be in USD, and it's unlikely that GOOGL's minimum price was \$1 on this day

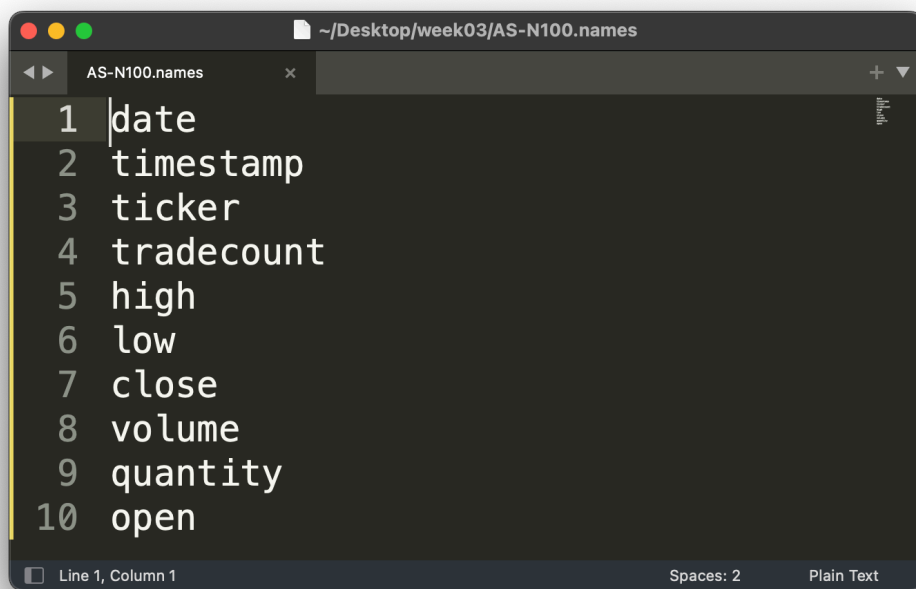
To Do

- What is the issue with this?
- Compare it with the file we had last week (open one in notepad or something)

-
- By looking at the minimum of the column, we have detected a problem with the data: We discovered that the file is wrong (or, our understanding of the file is wrong)
 - Going back to the `.names` file and the `.tsv` file from the previous week, we can see that the `.names` file is wrong: The columns *open* and *tradedcount* are swapped in the `.names` file. We can fix this by assigning the correct `colnames`

```
colnames(stock_data)[c(4, 10)] <- c("open", "tradedcount")
google_stock <- stock_data[stock_data[, "ticker"] == "GOOGL", ]
print(min(google_stock[, "open"]))
```

```
[1] 744.42
```



The screenshot shows a text editor window titled "AS-N100.names" with a file path of "~/Desktop/week03/AS-N100.names". The editor contains a list of 10 fields, each on a new line, numbered 1 through 10 on the left margin. The fields are: date, timestamp, ticker, trade count, high, low, close, volume, quantity, and open. The status bar at the bottom indicates "Line 1, Column 1", "Spaces: 2", and "Plain Text".

```
1 date
2 timestamp
3 ticker
4 trade count
5 high
6 low
7 close
8 volume
9 quantity
10 open
```

-
- Min and max are extremely useful to make sure our understanding of a dataset is correct. For example, if a value must be positive, then if *min* is negative that means there was at least one negative value, which contradicts our understanding of the data
 - Mistakes like this can happen frequently with real data, always good to do some basic checks on a new dataset
 - Even if you collect the data yourself you should do checks like this!

Standard deviation

- Moving on to the other summary statistics. The standard deviation measures the spread of the values in the data (or, the extent to which they vary around the mean). For values $x_1, \dots, x_n$ the *sample* standard deviation is defined as:

$$s = \sqrt{\frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2}$$

- Depends on the sample $x_1, \dots, x_n$

$$s = \sqrt{\frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2}$$

- Here $\bar{x}$ is the mean of the values. If the standard deviation is zero, then all the values are the same. (**Discussion:** Can you prove this?)

Mean

- The mean, or average of the values $x_1, \dots, x_n$ is defined as follows:

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$$

- The mean is a “usual” value, will capture overall structure in the data, if the data is “normal”.

-
- For example, a stock price changes throughout the day. But, if someone asks “what was the stock price of GOOGL on 2015-11-17”, this could be a good answer:

```
print(mean(google_stock[, "open"]))
```

```
[1] 750.4808
```

- However, if you looked at the mean price for Gamestop in 2021, the mean might not tell the whole story (because huge variability and strange patterns)

-
- The median is the number such that half the values are above the number and half the values are below the number. Here’s the formal definition:

$$\text{median} = \begin{cases} x_{(\frac{n+1}{2})} & \text{if } n \text{ is odd.} \\ \frac{x_{(n/2)} + x_{(\frac{n}{2}+1)}}{2} & \text{if } n \text{ is even.} \end{cases}$$

- Here $x_{(i)}$ is an *order statistic*: the i -th value in an ordered enumeration of the values from smallest to largest

Median

- The median is also known as the 50th percentile. The q -th percentile is the number such that a q -th proportion of the values are above the number.

Discussion: How would you define formally the 75-th percentile as an equation, as is done in the previous slide for the median?

Mean vs median

- The median is also a “usual” value. It is not affected by outliers:

```
x <- c(1, 7, 2, 9)
example <- data.frame()
for (v in c(9, 99, 999)) {
  x[4] <- v
  example <- rbind(example, c(median(x), mean(x), max(x)))
}
colnames(example) <- c("median", "mean", "max")
kable(example)
```

median	mean	max
4.5	4.75	9
4.5	27.25	99
4.5	252.25	999

```

x <- c(1, 7, 2, 9)
example <- data.frame()
for (v in c(9, 99, 999)) {
  x[4] <- v
  example <- rbind(example, c(median(x), mean(x), max(x)))
}
colnames(example) <- c("median", "mean", "max")
kable(example)

```

median	mean	max
4.5	4.75	9
4.5	27.25	99
4.5	252.25	999

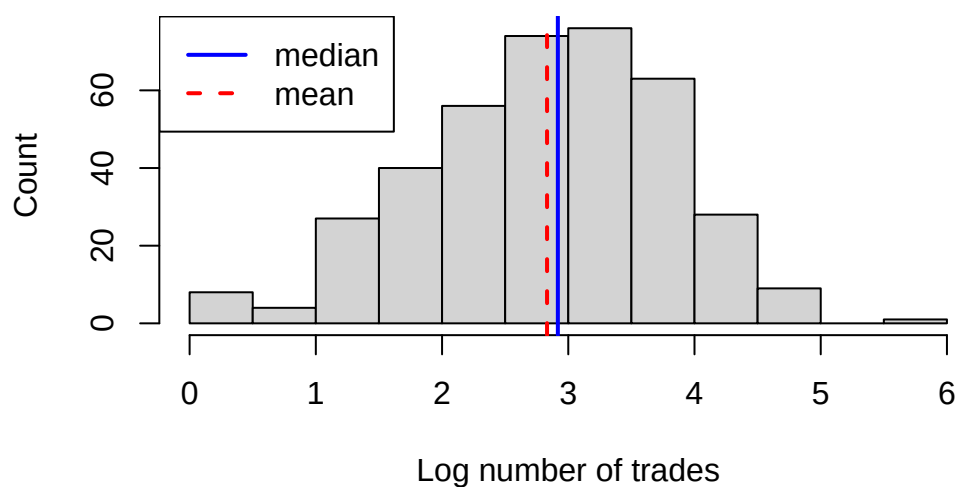
- The median and mean will be close when the data are unimodal and symmetric.

```

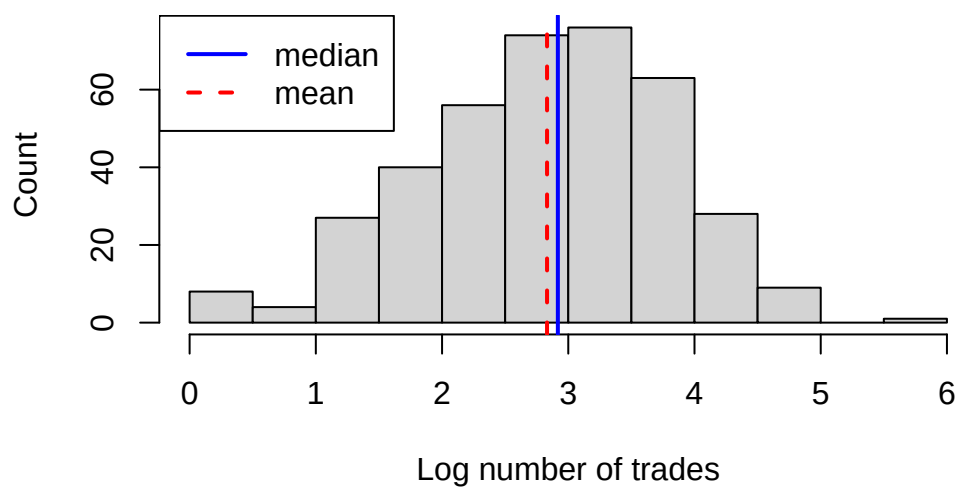
xs <- log(google_stock$tradedcount)
hist(xs,
  main = "GOOGL minutely on 2015-11-17", ylab = "Count",
  xlab = "Log number of trades"
)
abline(
  v = c(median(xs), mean(xs)), col = c("blue", "red"),
  lty = c(1, 2), lwd = c(2, 2)
)
legend("topleft", legend = c("median", "mean"), col = c(
  "blue",
  "red"
), lty = c(1, 2), lwd = c(2, 2))

```

GOOGL minutely on 2015-11-17



GOOGL minutely on 2015-11-17



```
xs <- google_stock$volume
hist(xs,
      main = "GOOGL minutely on 2015-11-17", ylab = "Count",
      xlab = "Number of trades")
```

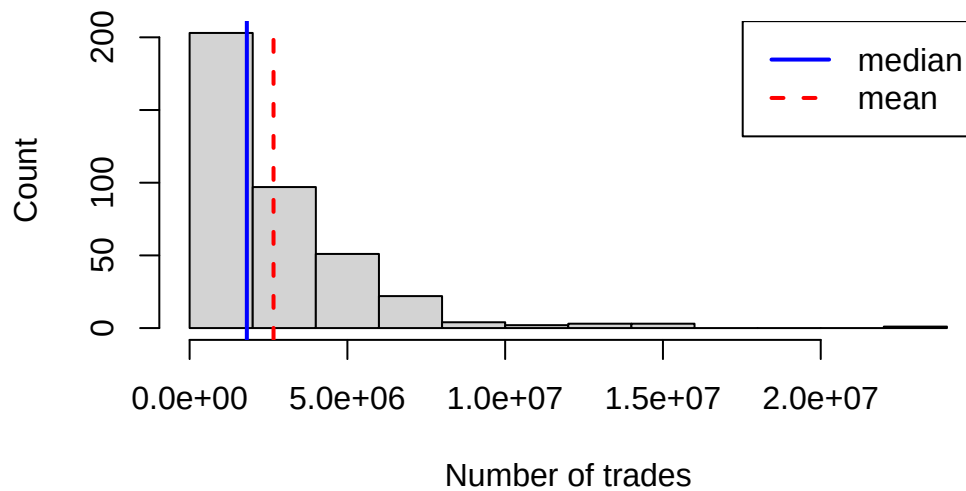


```

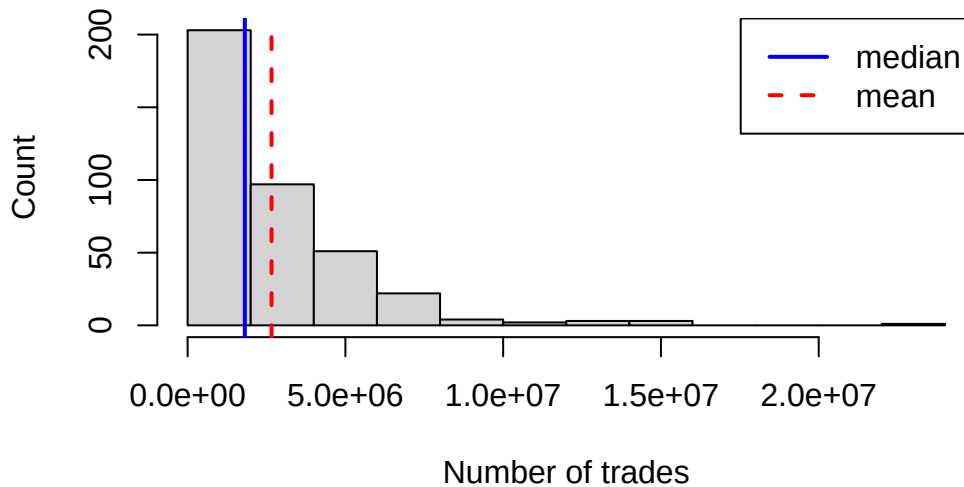
)
abline(
  v = c(median(xs), mean(xs)), col = c("blue", "red"),
  lty = c(1, 2), lwd = c(2, 2)
)
legend("topright", legend = c("median", "mean"), col = c(
  "blue",
  "red"
), lty = c(1, 2), lwd = c(2, 2))

```

GOOGL minutely on 2015-11-17



GOOGL minutely on 2015-11-17



Histograms

- Here we've looked at histograms, which are a basic tool of data visualization. Like summary statistics, they can be used to verify that our understanding of a dataset is correct, and to answer basic questions.
- A histogram of the values $x_1, \dots, x_n$ is a function that partitions the range of the values into equally spaced bins. The value of the histogram function is constant over the bins, and is given by the number of values that fall in that bin (left inclusive).

Histograms

- Formally a histogram with bin edges $a_1, \dots, a_k$ (i.e., the first bin is $[a_1, a_2)$ and the last bin is $[a_{k-1}, a_k]$) is defined as follows:

$$f(x) = \sum_{j=1}^{k-1} I(x, a_j, a_{j+1}) \cdot \sum_{i=1}^n I(x_i, a_j, a_{j+1})$$

- Histogram shows the range of values, can easily see min/max

$$f(x) = \sum_{j=1}^{k-1} I(x, a_j, a_{j+1}) \cdot \sum_{i=1}^n I(x_i, a_j, a_{j+1})$$

- Here $I(\cdot, a, b)$ is the indicator function for the set $[a, b)$ (i.e., it is 1 if its argument is in $[a, b)$ and zero otherwise). **Discussion:** Is this exactly right? What about the right most bin, shouldn't it be right and left inclusive?

Random variables

- The histogram is closely linked to a statistical object known as the *probability density function* or *pdf*
- In statistics, we often view observed values $x_1, \dots, x_n$ as random variables. We can view these as measurements.
- If these measurements are the results of performing replicates of the same experiment, then we say $x_1, \dots, x_n$ are independent and identically distributed
- (Probably) every statistical analysis you've seen makes this assumption!

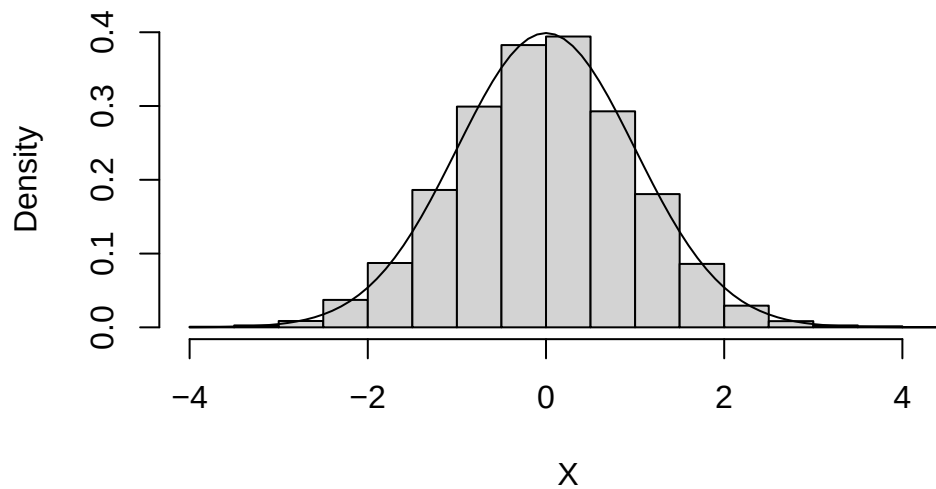
Normal distribution

- Based on assumptions about the experiment, a random variable x may have specific statistical properties. For example, we might assume that x is *normally distributed* with mean μ and standard deviation σ
- We would denote this $x \sim \mathcal{N}(\mu, \sigma)$. In this case, the probability that x lies in an interval $[a, b]$ is given by the integral from a to b of the *pdf* for the normal distribution with mean μ and standard deviation σ

Normal distribution

```
set.seed(240)
hist(rnorm(10000),
     freq = FALSE, ylim = c(0, .4), breaks = 25,
     xlab = "X", main = "Histogram of iid normal random variables"
)
xs <- seq(-4, 4, by = 0.1)
lines(xs, dnorm(xs))
```

Histogram of iid normal random variables



Normal distribution

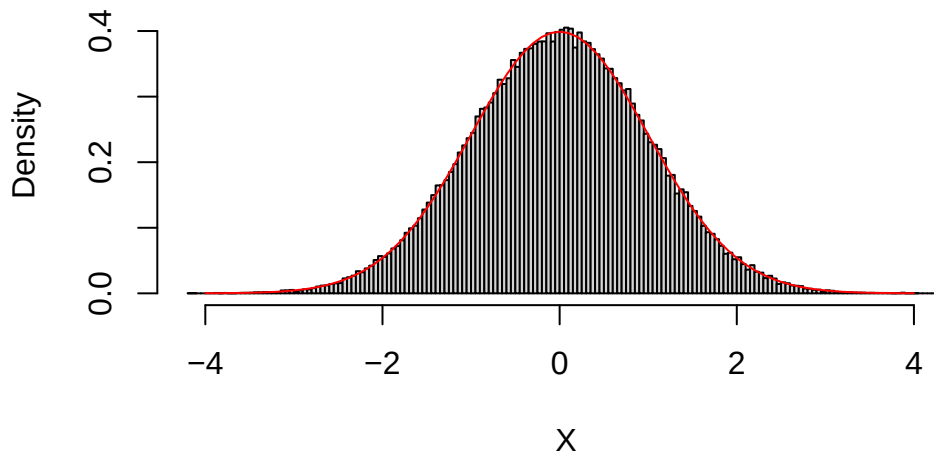
- If we integrate the curve between two bin edges, we get the probability that a standard normal random variable takes a value in that bin
- If we look at the height of the histogram over that bin, we get the proportion of random variables that actually fell in that bin. We can see that the probability and proportion are similar

To Do

- Use `?hist` to see how to increase the number of bins, and recreate the previous histogram
- Increase the number of samples also

-
- If we make the bins smaller and smaller it would get closer to this curve

Histogram of iid normal random variables



Data Visualization

Goals of Data Viz

- Visualizing data is an important but challenging skill in data science, could be a whole course!
- Things to remember:
 - Who is the audience for your visualization? What do they know about the data
 - What do you want them to take away from it? What is the goal
- Visualization is somewhat opinionated, but you can normally tell a good/bad figure when you see it

Things to keep in mind

- Don't overcomplicate things, eg
 - Don't include multiple y-axes
 - Don't include unnecessary labels, legends
- Pie charts are never the best way to convey information
- Make the key information clear, easy to read
- If using color make it color-blind friendly

Using ggplot

- ggplot a specific package for doing data visualization
 - Known as the Grammar of graphics
- Syntax for creating plots somewhat different if you haven't seen it before
- Not required but one of the big advantages to using R for data science

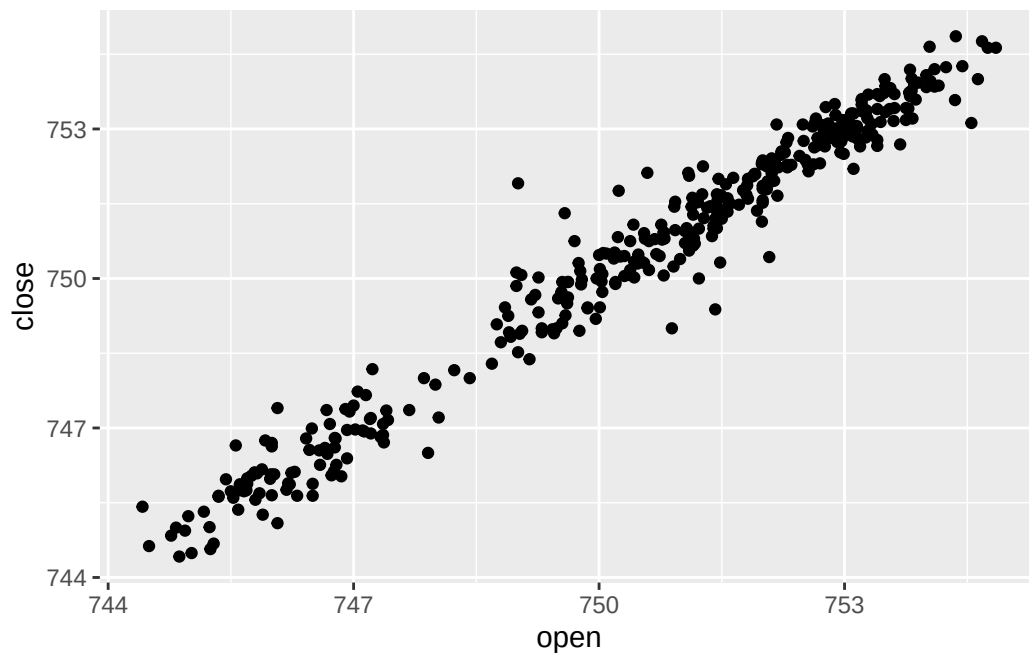
The ggplot framework (briefly)

- ggplot creates a figure with the following syntax

```
ggplot(data, aes(x_variable, y_variable)) +  
  geom_something() +  
  additional (extra) options
```

- `ggplot(data, aes(x_variable, y_variable))` specifies the data you will use, what variables in that data give the x and y axis
- `geom_` gives you the type of plot (scatter, line, histogram, bar)
- Add layers to the plot using `+`, unique to ggplot

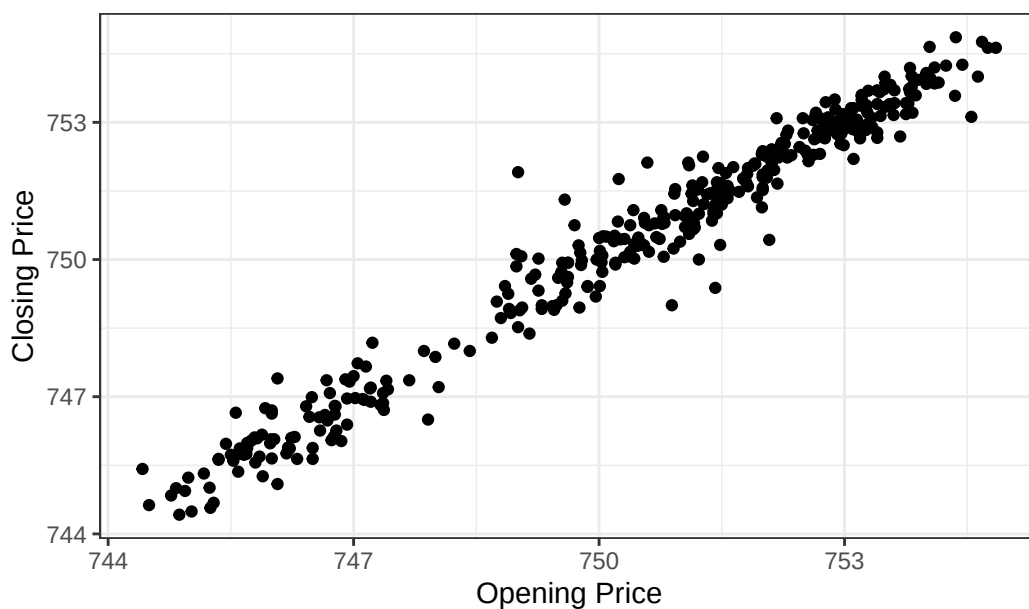
```
## library ggplot2  
ggplot(google_stock, aes(open, close)) +  
  geom_point()
```



- Many many ways you can customise these plots to make them cleaner, more informative

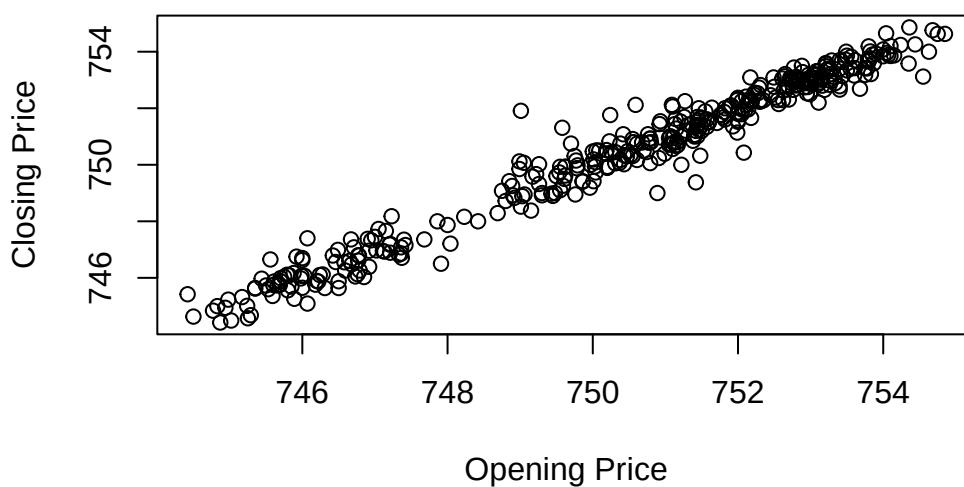
```
ggplot(google_stock, aes(open, close)) + geom_point() +  
  labs(x = "Opening Price", y = "Closing Price",  
        title = "Google Stock Price, November 17th 2015") +  
  theme_bw()
```

Google Stock Price, November 17th 2015



```
plot(x = google_stock$open, y = google_stock$close,  
     xlab = "Opening Price", ylab = "Closing Price",  
     main = "Google Stock Price, November 17th 2015")
```

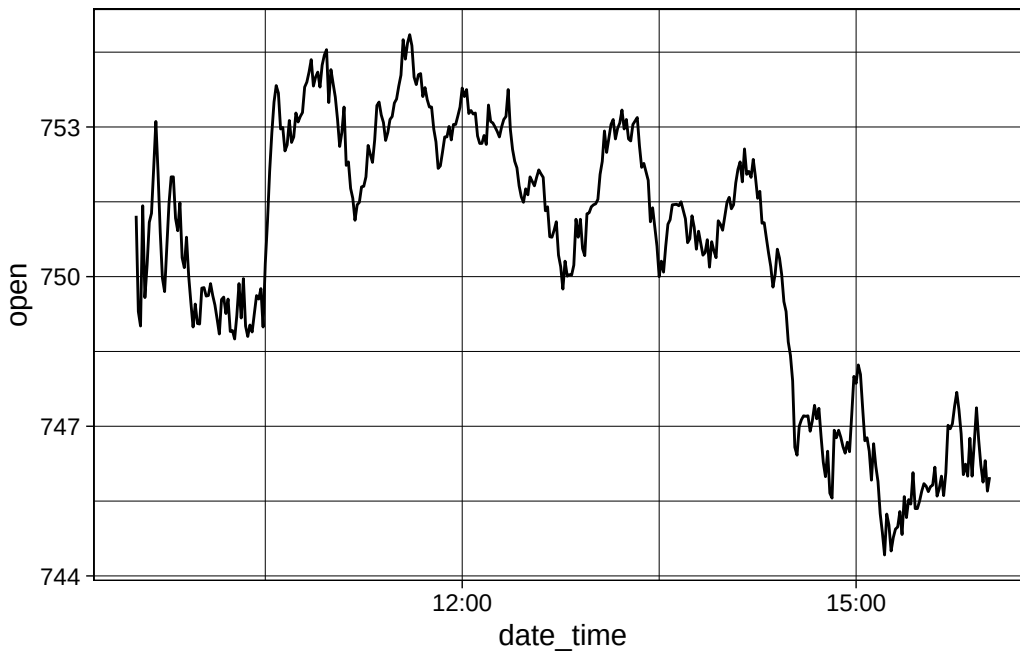
Google Stock Price, November 17th 2015



Line Plots (saw this last week)

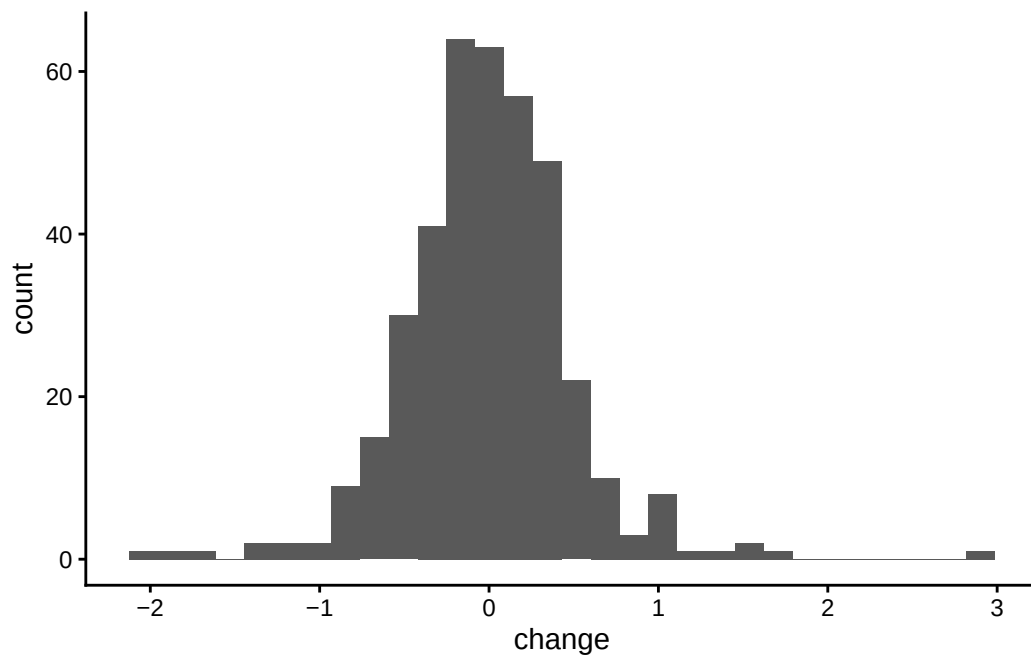
```
google_stock$date_time <- as.POSIXct(strptime(paste(
  "2015-11-17",
  google_stock$timestamp
), "%Y-%m-%d %H:%M:%S"))

ggplot(google_stock, aes(x = date_time, y = open)) +
  geom_line() + theme_linedraw()
```



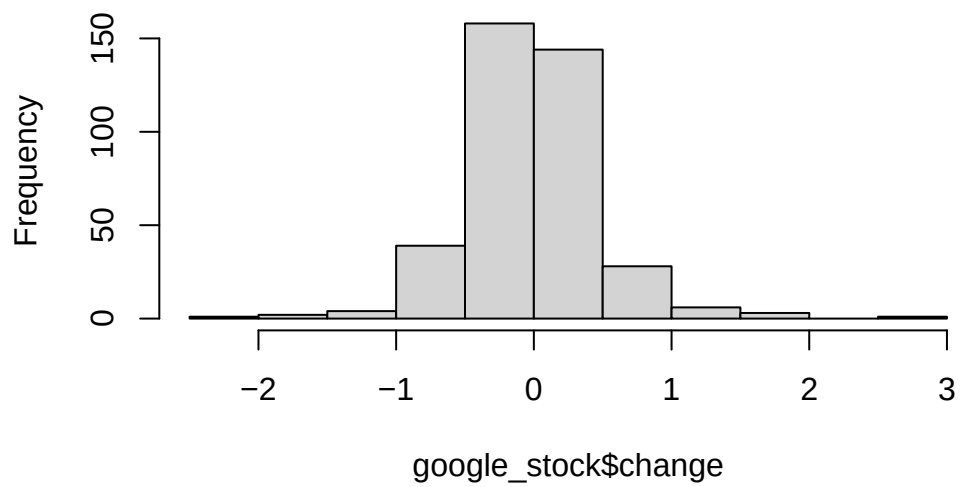
Histograms

```
google_stock$change <- google_stock$close - google_stock$open
ggplot(google_stock, aes(change)) + geom_histogram() + theme_classic()
```



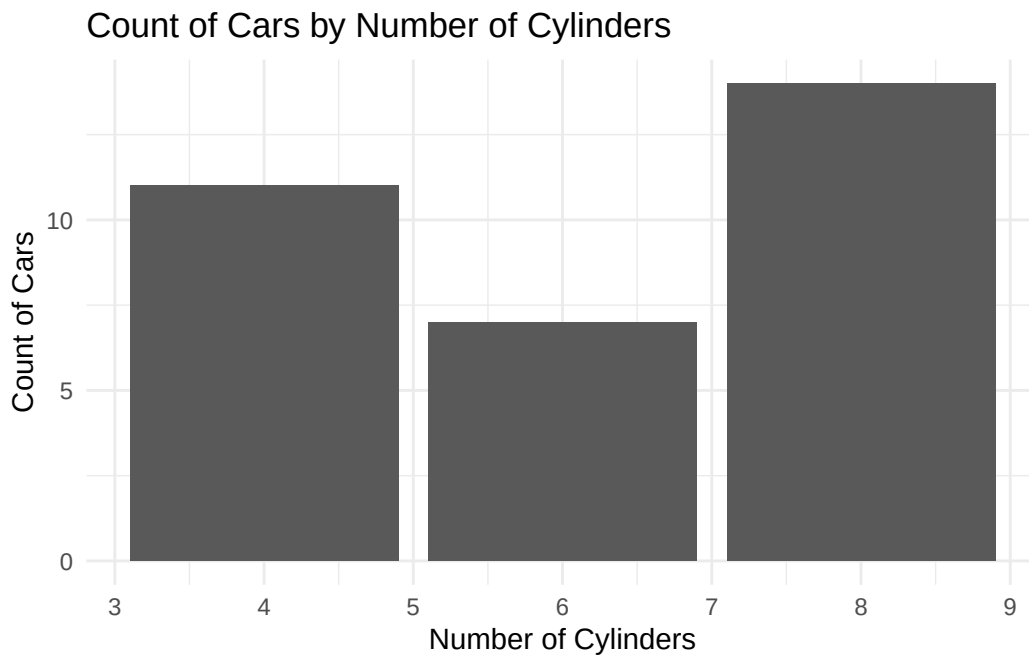
```
hist(google_stock$change)
```

Histogram of google_stock\$change



Bar Plots

```
ggplot(data = mtcars, aes(x = cyl)) + geom_bar() +  
  labs(title = "Count of Cars by Number of Cylinders",  
        x = "Number of Cylinders", y = "Count of Cars") +  
  theme_minimal()
```

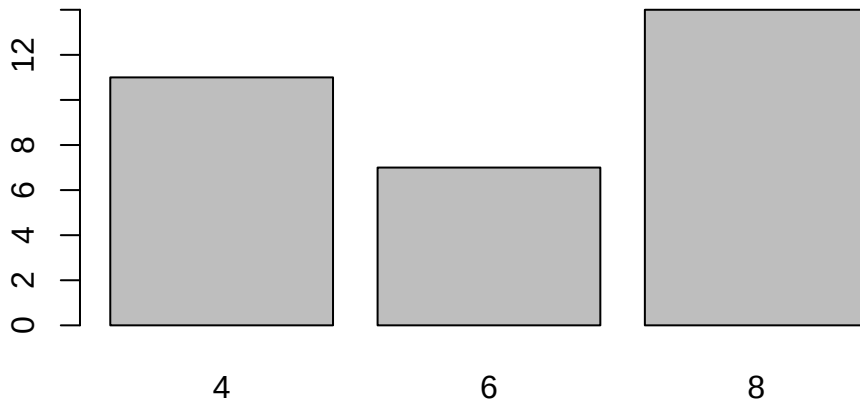


Base R version

```
cyl_counts <- table(mtcars$cyl)  
cyl_counts
```

```
 4  6  8  
11  7 14
```

```
barplot(cyl_counts)
```



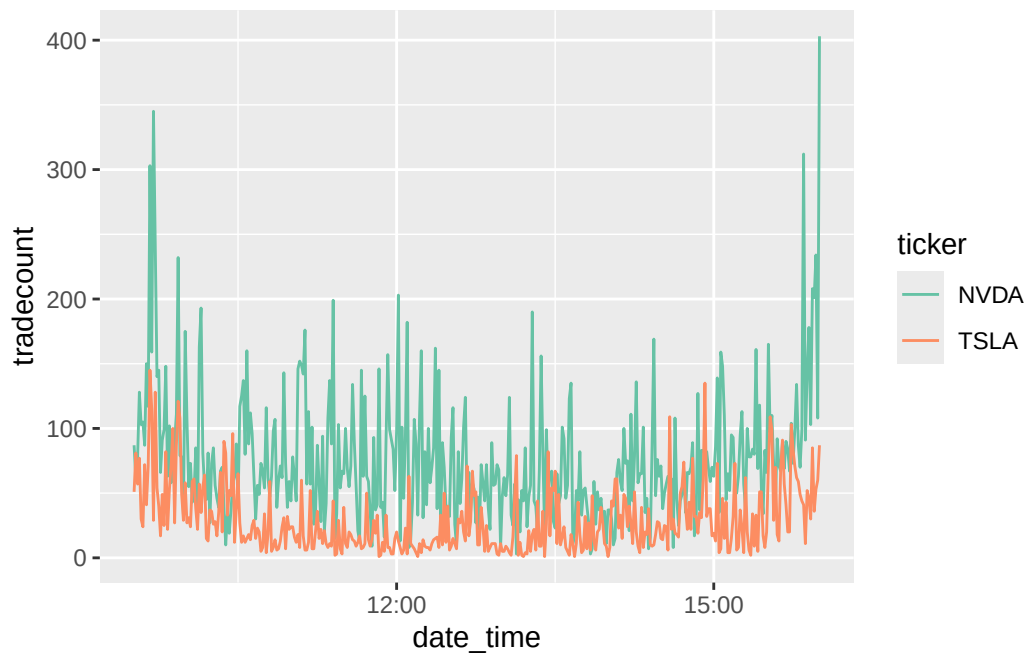
Incorporating Color

- Making graphics colorblind friendly particularly easy in ggplot
- Quick example using RColorBrewer package

```
#library(RColorBrewer)
tech <- c("TSLA", "NVDA")
tech_stocks <- stock_data[stock_data[, "ticker"] %in% tech, ]

tech_stocks$date_time <- as.POSIXct(strptime(paste(
  "2015-11-17",
  tech_stocks$timestamp
), "%Y-%m-%d %H:%M:%S"))
```

```
ggplot(tech_stocks, aes(date_time, tradecount, colour = ticker)) +
  geom_line() +
  scale_color_brewer(palette = "Set2")
```



Advanced R 1: Functions

- Here's a function to find the maximum value of a vector:

```
my_max <- function(v) {  
  result <- -Inf  
  for (i in 1:length(v)) {  
    if (v[i] > result) {  
      result <- v[i]  
    }  
  }  
  return(result)  
}  
print(my_max(c(1, 7, 2, 9)))
```

[1] 9

- Functions are used when we might want to use the same piece of code many times in different contexts

Advanced R 1: Functions

```
my_max <- function(v) {  
  ...  
}
```

- A function is defined using the keyword **function** followed by a round parenthesis enclosed list of parameters. The function body is enclosed in curly braces

Advanced R 1: Functions

```
...  
result <- -Inf  
for (i in 1:length(v)) {  
  if (v[i] > result) {  
    result <- v[i]  
  }  
}  
...  
}
```

- Function body looks at the parameter **v**, max value collected in the variable **result** which starts at $-\infty$
- Look at each element of **v** and check if it's bigger than **result**. If so update the value of **result** to be that element of **v**

Advanced R 1: Functions

```
...  
return(result)  
}
```

- A function can have a return value (specified in round parentheses after the keyword **return**)
- The function will “evaluate” to the returned value. For example, **x <- my_max(v)** will update the variable **x** so that it contains the value of **my_max** applied to the vector **v**

Advanced R 2: Lists

- Suppose I want a vector of length two, such that the first element is a vector of length two and the second element is a vector of length two:

```
v1 <- c(1, 7)
v2 <- c(2, 9)
v3 <- c(v1, v2)
print(v3)
```

```
[1] 1 7 2 9
```

```
print(length(v3))
```

```
[1] 4
```

- This didn't work. We got a vector of length 4

Advanced R 2: Lists

- The solution is to use a *list*. Along with data frames and vectors, lists are one of the basic data types in R:

```
v1 <- c(1, 7)
v2 <- c(2, 9)
v3 <- list(v1, v2)
print(v3)
```

```
[[1]]
[1] 1 7
```

```
[[2]]
[1] 2 9
```

```
print(length(v3))
```

```
[1] 2
```

Advanced R 2: Lists

- Lists can be nested, and elements of lists can be referred to by name or by index. We refer to list elements using double square brackets

```
print(v3[[1]])
```

```
[1] 1 7
```

```
print(v3[[2]])
```

```
[1] 2 9
```

-
- We can create lists using the function `list` and listing the elements (if we want to refer by name, we can express the element as `key = value`)

```
v3 <- list(FIRST = v1, SECOND = v2)
print(v3[["FIRST"]])
```

```
[1] 1 7
```

```
print(v3[["SECOND"]])
```

```
[1] 2 9
```

Preview of JSON

- A flat file is provided with this week's slides: `libraries.json`. This is a flat file in a special format "JavaScript object notation". This file contains information about all branches of the Vancouver public library. We can read this file using the following code:

```
# install.packages("rjson")
library(rjson)
libraries <- fromJSON(file = "libraries.json")
print(class(libraries))
```

```
[1] "list"
```

- The data is stored as a list

Libraries

```
names(libraries)
```

```
[1] "exceededTransferLimit" "features" "type"
```

```
print(libraries[["features"]])
```

```
[[1]]
[[1]]$id
[1] 1323

[[1]]$type
[1] "Feature"

[[1]]$properties
[[1]]$properties$objectid
[1] 1323

[[1]]$properties$maptip
[1] "Library:South Hill\\n6076 Fraser St"

[[1]]$geometry
[[1]]$geometry$type
[1] "Point"

[[1]]$geometry$coordinates
[1] -123.09029 49.22951

[[2]]
[[2]]$id
[1] 1324

[[2]]$type
[1] "Feature"

[[2]]$properties
```

```
[[2]]$properties$objectid
```

```
[1] 1324
```

```
[[2]]$properties$maptip
```

```
[1] "Library:West Point Grey\\n4480 W 10th Av"
```

```
[[2]]$geometry
```

```
[[2]]$geometry$type
```

```
[1] "Point"
```

```
[[2]]$geometry$coordinates
```

```
[1] -123.20866 49.26354
```

```
[[3]]
```

```
[[3]]$id
```

```
[1] 1325
```

```
[[3]]$type
```

```
[1] "Feature"
```

```
[[3]]$properties
```

```
[[3]]$properties$objectid
```

```
[1] 1325
```

```
[[3]]$properties$maptip
```

```
[1] "Library:Britannia\\n1661 Napier St"
```

```
[[3]]$geometry
```

```
[[3]]$geometry$type
```

```
[1] "Point"
```

```
[[3]]$geometry$coordinates
```

```
[1] -123.07379 49.27565
```

```
[[4]]
```

```
[[4]]$id
```

```
[1] 1326
```

```
[[4]]$type  
[1] "Feature"
```

```
[[4]]$properties  
[[4]]$properties$objectid  
[1] 1326
```

```
[[4]]$properties$maptip  
[1] "Library:Carnegie\\n401 Main St"
```

```
[[4]]$geometry  
[[4]]$geometry$type  
[1] "Point"
```

```
[[4]]$geometry$coordinates  
[1] -123.10014 49.28106
```

```
[[5]]  
[[5]]$id  
[1] 1327
```

```
[[5]]$type  
[1] "Feature"
```

```
[[5]]$properties  
[[5]]$properties$objectid  
[1] 1327
```

```
[[5]]$properties$maptip  
[1] "Library:Champlain\\n7110 Kerr St"
```

```
[[5]]$geometry  
[[5]]$geometry$type  
[1] "Point"
```

```
[[5]]$geometry$coordinates  
[1] -123.04020 49.21909
```

```

[[6]]
[[6]]$id
[1] 1328

[[6]]$type
[1] "Feature"

[[6]]$properties
[[6]]$properties$objectid
[1] 1328

[[6]]$properties$maptip
[1] "Library:Collingwood\\n2985 Kingsway"

[[6]]$geometry
[[6]]$geometry$type
[1] "Point"

[[6]]$geometry$coordinates
[1] -123.04265 49.23624


[[7]]
[[7]]$id
[1] 1329

[[7]]$type
[1] "Feature"

[[7]]$properties
[[7]]$properties$objectid
[1] 1329

[[7]]$properties$maptip
[1] "Library:Dunbar\\n4515 Dunbar St"

[[7]]$geometry
[[7]]$geometry$type

```

```

[1] "Point"

[[7]]$geometry$coordinates
[1] -123.18564 49.24598


[[8]]
[[8]]$id
[1] 1330

[[8]]$type
[1] "Feature"

[[8]]$properties
[[8]]$properties$objectid
[1] 1330

[[8]]$properties$maptip
[1] "Library:Firehall\\n1455 W 10th Av"


[[8]]$geometry
[[8]]$geometry$type
[1] "Point"

[[8]]$geometry$coordinates
[1] -123.13759 49.26288


[[9]]
[[9]]$id
[1] 1331

[[9]]$type
[1] "Feature"

[[9]]$properties
[[9]]$properties$objectid
[1] 1331

[[9]]$properties$maptip

```

```
[1] "Library:Fraserview\\n1950 Argyle Drive"
```

```
[[9]]$geometry  
[[9]]$geometry$type  
[1] "Point"
```

```
[[9]]$geometry$coordinates  
[1] -123.06687 49.21966
```

```
[[10]]  
[[10]]$id  
[1] 1332
```

```
[[10]]$type  
[1] "Feature"
```

```
[[10]]$properties  
[[10]]$properties$objectid  
[1] 1332
```

```
[[10]]$properties$maptip  
[1] "Library:Hastings\\n2674 E Hastings St"
```

```
[[10]]$geometry  
[[10]]$geometry$type  
[1] "Point"
```

```
[[10]]$geometry$coordinates  
[1] -123.05006 49.28087
```

```
[[11]]  
[[11]]$id  
[1] 1333
```

```
[[11]]$type  
[1] "Feature"
```

```
[[11]]$properties
[[11]]$properties$objectid
[1] 1333
```

```
[[11]]$properties$maptip
[1] "Library:Kitsilano\\n2425 MacDonald St"
```

```
[[11]]$geometry
[[11]]$geometry$type
[1] "Point"
```

```
[[11]]$geometry$coordinates
[1] -123.16871 49.26472
```

```
[[12]]
[[12]]$id
[1] 1334
```

```
[[12]]$type
[1] "Feature"
```

```
[[12]]$properties
[[12]]$properties$objectid
[1] 1334
```

```
[[12]]$properties$maptip
[1] "Library:Mount Pleasant\\n1 Kingsway"
```

```
[[12]]$geometry
[[12]]$geometry$type
[1] "Point"
```

```
[[12]]$geometry$coordinates
[1] -123.10024 49.26432
```

```
[[13]]
[[13]]$id
```

```

[1] 1335

[[13]]$type
[1] "Feature"

[[13]]$properties
[[13]]$properties$objectid
[1] 1335

[[13]]$properties$maptip
[1] "Library:nécamat ct Strathconan\\n780 E Hastings St"

[[13]]$geometry
[[13]]$geometry$type
[1] "Point"

[[13]]$geometry$coordinates
[1] -123.08892 49.28083


[[14]]
[[14]]$id
[1] 1336

[[14]]$type
[1] "Feature"

[[14]]$properties
[[14]]$properties$objectid
[1] 1336

[[14]]$properties$maptip
[1] "Library:Terry Salman\\n4575 Clancy Loranger Way"

[[14]]$geometry
[[14]]$geometry$type
[1] "Point"

[[14]]$geometry$coordinates
[1] -123.10786 49.24424

```



```
[[15]]
[[15]]$id
[1] 1337

[[15]]$type
[1] "Feature"

[[15]]$properties
[[15]]$properties$objectid
[1] 1337

[[15]]$properties$maptip
[1] "Library:Renfrew\\n2969 E 22nd Av"
```

```
[[15]]$geometry
[[15]]$geometry$type
[1] "Point"

[[15]]$geometry$coordinates
[1] -123.04297 49.25243
```

```
[[16]]
[[16]]$id
[1] 1338

[[16]]$type
[1] "Feature"

[[16]]$properties
[[16]]$properties$objectid
[1] 1338

[[16]]$properties$maptip
[1] "Library:Central Branch\\n350 W Georgia St"
```

```
[[16]]$geometry
```

```
[[16]]$geometry$type  
[1] "Point"
```

```
[[16]]$geometry$coordinates  
[1] -123.1156 49.2797
```

```
[[17]]  
[[17]]$id  
[1] 1339
```

```
[[17]]$type  
[1] "Feature"
```

```
[[17]]$properties  
[[17]]$properties$objectid  
[1] 1339
```

```
[[17]]$properties$maptip  
[1] "Library:Joe Fortes\\n870 Denman St"
```

```
[[17]]$geometry  
[[17]]$geometry$type  
[1] "Point"
```

```
[[17]]$geometry$coordinates  
[1] -123.13609 49.28992
```

```
[[18]]  
[[18]]$id  
[1] 1340
```

```
[[18]]$type  
[1] "Feature"
```

```
[[18]]$properties  
[[18]]$properties$objectid  
[1] 1340
```

```
[[18]]$properties$maptip
[1] "Library:Kensington\\n1428 Cedar Cottage Mews"
```

```
[[18]]$geometry
[[18]]$geometry$type
[1] "Point"
```

```
[[18]]$geometry$coordinates
[1] -123.07536 49.24908
```

```
[[19]]
[[19]]$id
[1] 1341
```

```
[[19]]$type
[1] "Feature"
```

```
[[19]]$properties
[[19]]$properties$objectid
[1] 1341
```

```
[[19]]$properties$maptip
[1] "Library:Kerrisdale\\n2112 W 42nd Av"
```

```
[[19]]$geometry
[[19]]$geometry$type
[1] "Point"
```

```
[[19]]$geometry$coordinates
[1] -123.15711 49.23319
```

```
[[20]]
[[20]]$id
[1] 1342
```

```
[[20]]$type
[1] "Feature"
```

```
[[20]]$properties
[[20]]$properties$objectid
[1] 1342
```

```
[[20]]$properties$maptip
[1] "Library:Marpole\\n8386 Granville St"
```

```
[[20]]$geometry
[[20]]$geometry$type
[1] "Point"
```

```
[[20]]$geometry$coordinates
[1] -123.1401 49.2104
```

```
[[21]]
[[21]]$id
[1] 1343
```

```
[[21]]$type
[1] "Feature"
```

```
[[21]]$properties
[[21]]$properties$objectid
[1] 1343
```

```
[[21]]$properties$maptip
[1] "Library:Oakridge\\n191-650 W 41st Av"
```

```
[[21]]$geometry
[[21]]$geometry$type
[1] "Point"
```

```
[[21]]$geometry$coordinates
[1] -123.1172 49.2327
```

Libraries

- Using `names` and `print` and by examining the flat file, we can index deep into the nested lists to extract the data: The names and GPS coordinates of the library branches:

```
print(libraries[["features"]][[2]][["properties"]]$maptip)
```

```
[1] "Library:West Point Grey\\n4480 W 10th Av"
```

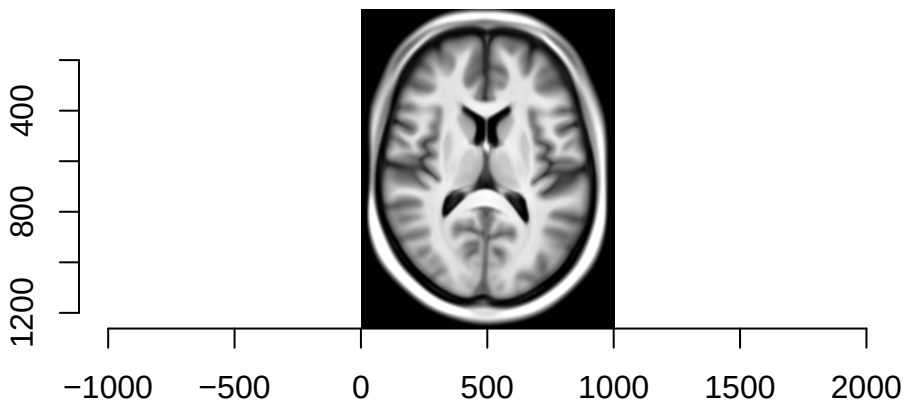
```
print(libraries[["features"]][[2]][["geometry"]]$coordinates)
```

```
[1] -123.20866 49.26354
```

Preview of images

- To load an image, we can use the `load.image` function from the package *imager*

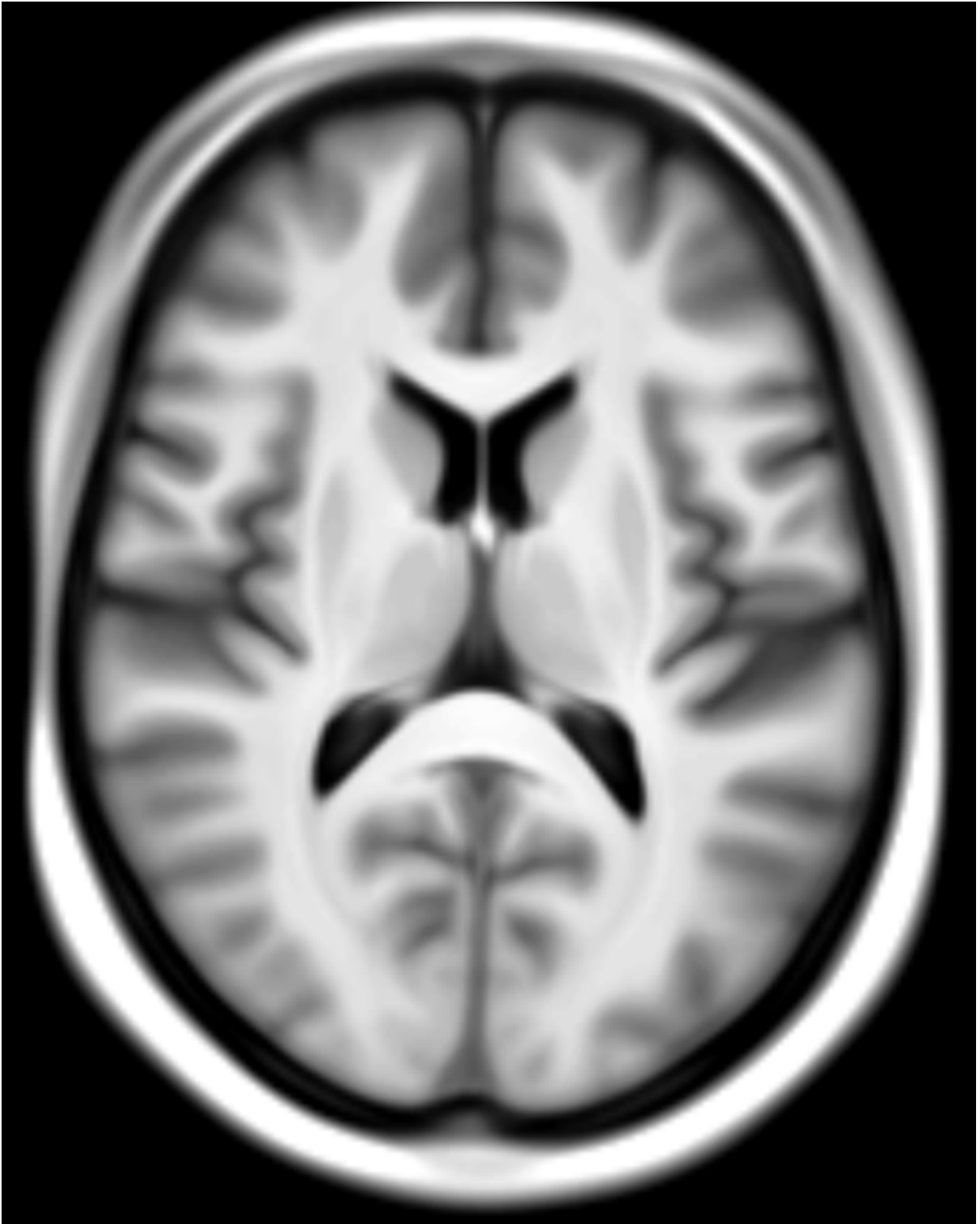
```
# install.packages("imager")
library(imager)
image <- load.image("Figure02.png")
plot(image)
```



Preview of images

- This is a slice from an averaged human brain MRI
- We can specify `axes = FALSE` to get just the image itself:

```
plot(image, axes = FALSE)
```



-
- This plot function lets us further interact with the display, as we saw with overlaying onto the histogram earlier in these slides. We'll add an 'X' to the exact centre of the image, with the text "Centre" next to it:

```
W <- dim(image)[1]
H <- dim(image)[2]
plot(image, axes = FALSE)
points(W / 2, H / 2, col = "red", pch = 4, lwd = 5)
text(W / 2 + 125, H / 2, "Centre", col = "red")
```

